1 Trynext Lifestyle — Technical Stack and Operations Handoff

Domain: `https://trynext.shop`

Brand: Trynext Lifestyle

Handoff date: 2026-10-30

Audience: owner, senior developers, release and operations maintainers

1.1 1. Executive summary

Trynext Lifestyle is a Bangladesh-focused custom-commerce platform. Customers browse the restored catalog, upload artwork, customize products in the Design Studio, add designs to cart, and complete checkout with the local commerce rules. Administrators manage catalog, mockups, orders, customers, settings, promotions, content, backups, automation, security, and operational health.

The current production baseline is healthy: the public homepage, readiness endpoint, product API, and categories API respond successfully. The restored public catalog is 69 products across 7 categories. Production runs with a primary API role and the selected recovery database. The Studio release provides compositor-first product previews, compact controls, single-tap selection, outside-frame dismissal, double-tap Edit, and direct Crop entry.

The system is operational, but it is not yet a true multi-database automatic failover system. An independent Neon project is not a replica simply because it contains the same schema. Safe transactional failover requires synchronized data, freshness evidence, promotion fencing, quota monitoring, and a tested rollback procedure.

1.2 2. High-level request flow

```
Customer browser
    ↓ relative /api/* requests
Cloudflare Pages frontend + Pages gateway
    ↓ primary API routing, timeout policy, caching/security headers
Render Node/Express API
    ↓ Drizzle ORM / node-postgres
Neon PostgreSQL primary
    ↓
orders, customers, catalog, mockups, admin, settings, sessions
```

Static frontend assets are delivered by Cloudflare Pages. API requests use relative paths so the site remains bound to `trynext.shop` instead of exposing an internal provider URL in the browser. The Pages gateway applies API routing and longer bounded budgets for long-running AI operations.

1.3 3. Frontend stack

Area	Technology	Responsibility
Application	React + TypeScript	Storefront, Design Studio, admin UI, routing and stateful views
Build	Vite	Development and production bundling
Styling	Tailwind CSS	Responsive utility styling and premium Studio/admin UI
Components	Radix UI and project components	Accessible dialogs, tabs, sliders, menus and primitives
Motion	Framer Motion	Controlled page and interaction transitions
Canvas	Konva.js / react-konva	Artwork selection, drag, resize, rotation and gesture interaction
PWA	Workbox/Vite PWA integration	Service worker and precached application assets
Icons	Lucide React	Consistent interface iconography
Browser image work	Canvas APIs and local processing	Validation, crop, extend, compositing and image preparation

1.3.1 Design Studio behavior

The Studio has three separate responsibilities:

1. **Visual authority:** `LiveCompositorPreview` renders the approved product compositor. The legacy SVG is not painted beneath an active compositor.
2. **Interaction authority:** Konva layers handle selection, movement, scaling, rotation, taps, double taps and touch gestures.
3. **Editing authority:** the upload/ImagePanel workflow handles background removal, upscale, auto-fix, AI reference editing, crop, and transparent extension.

Current interaction contract:

- One tap on artwork selects it and shows compact controls.
- A tap or press outside the selected artwork's actual rotated frame clears the selection.
- The transparent Konva hitbox does not override the visible frame boundary.
- A delayed tap produced after a long press cannot immediately restore the cleared controls.
- Double tap/double click on image artwork opens the full Edit/Image Tools workflow.
- Contextual Crop opens the crop dialog directly.

- On mobile, Crop keeps the ImagePanel mounted so the fixed crop dialog can open instead of being unmounted by closing the sheet.
- Crop and Edit are separate actions; Crop does not silently change the Edit workflow.

1.4 4. Backend stack

Area	Technology	Responsibility
Runtime	Node.js	API process and scheduled operational work
HTTP API	Express	Routes, middleware, authentication, health, catalog, checkout and admin APIs
Database access	Drizzle ORM	Typed schema access and query composition
PostgreSQL driver	`pg` / node-postgres	Connection pools and database connectivity
Validation	Zod / drizzle-zod	Request, payload and schema validation
Tests	Vitest + Supertest	Unit, route, API, integration-style and regression tests
Storage	R2/object-storage-compatible adapter	Uploads, artwork, generated assets and mockup files
Auth	Protected admin/session middleware with TOTP support	Admin authentication, sessions, revocation and role boundaries
Observability	Health endpoints, activity log, request metrics, structured logs	Runtime diagnosis and operational evidence

1.4.1 API request behavior

The product API is paginated:

```
GET /api/products?page=1&limit=24&includeTotal=true
```

The default page size is 24. Explicit requests are capped at 100. The response includes the product array and pagination metadata such as `total`, `page`, `limit`, and `totalPages`. The restored catalog can be requested in several pages or with a bounded `limit=100` request; the API does not force every visitor to download the entire catalog.

Typical public requests include:

- `/api/products`
- `/api/categories`

- `/api/mockups`
- `/api/settings` where public settings are allowed
- `/api/health/readiness`
- `/api/health/liveness`

Protected operations include authentication, admin actions, checkout, order creation, payment proof uploads, customer records, settings mutations, backup actions, scheduler actions, and AI provider requests.

1.4.2 AI request handling

AI generation and reference-edit calls receive a bounded 45-second gateway budget instead of the ordinary short API timeout. The browser does not receive provider secrets. Client operations retain timeouts and preserve the original artwork when an enhancement fails or times out.

1.5 5. Data model and database roles

The authoritative transactional database contains the operational source of truth for:

- products, categories, variants and catalog settings;
- mockup records and runtime asset metadata;
- customers, addresses, orders and order items;
- payment status and Bangladesh-specific COD rules;
- admin users, sessions, roles, activity logs and settings;
- reviews, notifications, campaigns and operational records.

1.5.1 Production primary

The recovery record identifies **Neon project `trynex-main-backup`, branch `main`** as the selected production database source. The public readiness response currently reports `runtimeRole: primary`, with database and catalog healthy.

The exact private Render environment value must be confirmed in Render rather than copied into chat or documentation.

1.5.2 Standby and backup terminology

A second database must be labelled accurately:

- **Transactional standby:** synchronized and safe for promotion after freshness verification.
- **Read replica:** safe only for approved reads; never for orders, payments, customers or admin writes.
- **Catalog snapshot:** useful for comparison/recovery but not safe for live transactional failover.
- **Archive/export:** recovery material, not a runtime destination.

The application now follows explicit configured candidate order. It no longer decides that an independent database is authoritative merely because it has more rows. Schema presence and row counts are diagnostics, not replication proof.

1.6 6. Bangladesh commerce rules

The platform preserves the local commerce model, including:

- 25% advance required for Cash on Delivery orders;
- remaining COD amount handled according to the configured checkout/order workflow;
- Bangladesh delivery and contact information fields;
- payment-proof handling where configured;
- order status tracking and admin review.

Payment and order mutations must remain primary-only and should be treated as idempotent operations. A network timeout is not proof that an order failed; duplicate prevention and server-side order status remain authoritative.

1.7 7. Admin panel powers

The protected admin area includes operational and commercial capabilities such as:

- dashboard and live health;
- products, variants, categories, prices, stock, colors and featured flags;
- mockup records, asset coverage, render status and validation;
- order review, status updates, payment verification and fulfillment workflows;
- customer lookup and operational support tools;
- settings, Studio color palettes, pricing and customization fees;
- reviews, newsletter, blog, SEO, promos, referrals and page content;
- database-cluster health and candidate status;
- backup and restore operations where configured;
- automation/scheduler status;
- security, sessions, admin roles, activity logs and TOTP-related controls;
- AI developer/diagnostic surfaces for approved provider operations.

A separate admin frontend could isolate UI deployment risk, but it cannot make database-backed admin writes work while the transactional database is down. The safe architecture is a separate admin route/auth boundary, optionally a separate Pages project, while transactional admin actions continue using the authoritative API/database.

1.8 8. Hosting and deployment

1.8.1 Cloudflare Pages

Cloudflare Pages serves the production storefront and gateway. The gateway is responsible for API origin routing, bounded timeouts, and safe request behavior. The public domain is `trynext.shop`.

1.8.2 Render

The active production service is:

- `trynex-lifestyle-main-render`
- repository: `georgelsmith333-hub/New-Trynext`
- branch: `main`
- region: Oregon
- runtime: Node
- current role: primary API
- current status: active/not suspended

The older `trynex-lifestyle` service is owner-suspended and is not a configured production standby.

1.8.3 Neon

Neon provides PostgreSQL projects and branches. The currently selected production source is documented as `trynex-main-backup/main`. A real standby must be continuously/frequently synchronized and monitored for freshness; a manually copied database is not enough for automatic transactional failover.

1.9 9. Resilience policy

The correct policy is:

- **80% provider usage:** warning alert, check synchronization and prepare capacity.
- **90% provider usage:** critical alert, stop nonessential jobs and verify standby readiness.
- **Primary outage:** promote only a known-current synchronized standby.
- **No verified standby:** fail closed for risky transactional writes rather than writing to a stale independent database.

Automatic migration at 80–90% cannot safely be invented by the application process. It requires provider usage metrics, a durable sync job, freshness watermark, promotion fencing, secrets, and rollback. The repository documents these boundaries and removes row-count-based authority selection, but provider quota alerts and replication must be configured in Neon/Render/Cloudflare.

The current production readiness response reports `backupSyncEnabled: false`. That is an explicit reminder that backup synchronization is not yet verified as an active runtime capability.

1.10 10. Security and privacy boundaries

- Never put database URLs, Neon API keys, provider tokens, payment credentials or customer data in source control or chat.
- Customer and order data are transactional records and should be accessed only for an approved operational purpose.
- Admin routes require authentication and role checks.
- Activity logging should cover sensitive admin mutations.
- Provider secrets belong in Render/Cloudflare secret stores.
- The four Neon API keys previously pasted into chat history should be revoked and rotated by the owner.

1.11 11. Current release evidence

At the handoff point:

- live homepage: HTTP 200;
- readiness: HTTP 200, `status=ok`, `db=true`, `catalog=true`, `runtimeRole=primary`;
- live catalog: 69 products and 7 categories;
- API test suite previously passed: 34 files / 249 tests;
- storefront correction pass: 36 files / 234 tests;
- storefront typecheck: passed;
- storefront production build: passed;
- Studio correction tests: 18 focused tests passed;
- PR #83 checks: typecheck/build, tests, security scan and Cloudflare Pages passed before the final Studio correction commit;
- final Studio correction commit: `cb70f8e`;
- PR #83: open and mergeable pending checks for the latest commit.

1.12 12. Owner runbook

1.12.1 If the storefront appears offline

1. Check `https://trynext.shop/api/health/readiness`.

2. Confirm `db=true` and `catalog=true`.
3. Check Render service status and recent events.
4. Check whether the active database is reachable and within quota.
5. Do not point the API to an arbitrary database based on product count.
6. If no synchronized standby is verified, protect order/admin writes instead of silently promoting stale data.

1.12.2 If Studio controls appear again after dismissal

1. Confirm the deployed commit contains the Studio correction pass.
2. Test a tap inside the artwork frame, outside the artwork frame, and a press/hold outside the frame.
3. Confirm the selection overlay stays hidden after release.
4. Confirm toolbar buttons preserve selection.
5. Confirm double tap opens Edit and Crop opens the crop dialog directly.

1.12.3 If Crop appears not to work

1. Select an image layer.
2. Tap contextual Crop once.
3. Confirm the fixed Image Tools dialog appears with Crop selected.
4. Drag the frame or a handle.
5. Apply crop and confirm the image is replaced while its product position remains stable.
6. Cancel and confirm the original image remains unchanged.

1.13 13. Remaining owner/provider tasks

- Verify the private Render database target is `trynex-main-backup/main`.
- Establish a genuinely synchronized transactional standby.
- Configure 80% and 90% quota alerts.
- Decide whether a separate admin Pages frontend is worth the operational complexity.
- Perform real-phone Studio acceptance testing after the latest production deployment.
- Rotate the four previously exposed Neon API keys.